Reviewer Pack — Minimal Rank of Entropic Complexity of Boolean Functions vs …

Entry 56f3ed73e3fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 20:47:13 UTC

Minimal Rank of Entropic Complexity of Boolean Functions vs BP_ReadTwice Circuit Size

Entry ID: `56f3ed73e3fe` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 20:47:13 UTC

1. Conjecture

Field A (mathematical branch): Information Theory (Entropy and Information Measures) **Field B** (complexity object): Complexity Theory: Branching Program (BP_ReadTwice) Complexity

Statement:

> ['For any boolean function f, the entropic complexity E(f) is upper-bounded by a constant times log(size of BP for f)) and lower-bounded by Ω(log(size of BP for f)))', 'Entropically speaking, if the BP_read_twice circuit size for f is s, then 1 ≤ E(f) ≤ log(s + 1)', 'For all boolean functions f with n ≤ 40 variables, this relation holds.']

Rationale (proposer's reasoning):

> ['Entropy provides a measure of information content that could potentially reveal structural properties of computational objects like BP_read_twice circuits.', 'Previous work has shown that entropy can be used to analyze complexity classes like IP and BPP, suggesting its utility in understanding other complexity measures as well.', 'The proposed conjecture aims to connect the abstract concept of entropy with a specific complexity-theoretic object, providing new insights into the structure of BP_read_twice circuits.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9752cac4231c0a65`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all boolean functions f with n ≤ 40 variables, the entropic complexity E(f) falls within the range [1, log(s + 1)] where s is the BP_read_twice circuit size for f, and falsified if any seed produces an E(f) outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "entropic complexity" AND "BP_ReadTwice circuit size" - "boolean function" AND "upper bound" AND "log(size of BP)" - "lower bound" AND "Ω(log(size of BP))" AND "entropic complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def bp_read_twice_circuit_size(f):
        n = int(math.log2(len(f)))
        # Simplified model of BP_read_twice circuit size
        return 2 * n + 3

    def entropic_complexity(f):
        # Simplified entropy calculation (logarithm base 2)
        return math.log2(1 / sum(1 for x in f if x == 0) + 1)

    results = []
    for _ in range(30):  # Ensure at least 30 instances per seed
        n = random.randint(5, 40)
        f = generate_boolean_function(n)
        s = bp_read_twice_circuit_size(f)
        E_f = entropic_complexity(f)

        if not (1 <= E_f <= math.log2(s + 1)):
            return {
                "metric_name": "Entropic Complexity vs BP ReadTwice Circuit Size",
                "metric_value": E_f,
                "instances_tested": 1,
```

```
                "conjecture_holds": False,
                "counterexample": f"n={n}, s={s}, E(f)={E_f}"
            }

        results.append(E_f)

    return {
        "metric_name": "Entropic Complexity vs BP ReadTwice Circuit Size",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 1 and r <= math.log2(s + 1)) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r < 1 or r > math.log2(s + 1) for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not (1 <= result
        print(f"RESULT: FALSIFIED counterexample=\"out of range\" first_failing_seed={first_failir
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which prevents us from verifying whether all boolean functions with n ≤ 40 variables have entropic complexity E(f) within the range [1, log(s + 1)]. | next: Run the test again to ensure it completes without crashing or timing out and verify the results for all boolean functions with n ≤ 40 variables.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11716
2	preregistration	ollama_remote	glm4:latest	0	0	5747
3	novelty	ollama_remote	glm4:latest	0	0	4727
4	novelty	ollama_remote	glm4:latest	0	0	5433
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14998
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8610
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10081
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9413
9	judge	ollama_remote	glm4:latest	0	0	23534

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94260 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/56f3ed73e3fe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/56f3ed73e3fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/56f3ed73e3fe.tar.gz` (if generated)